

Redemption Service — Design Document

The Redemption — Infrastructure Design

Accor APAC | ap-southeast-1 | Cloud Engineer Assessment

Overview

The Redemption is a hotel loyalty point deduction service handling steady baseline traffic with sudden 10x spikes during flash sales. The infrastructure is on AWS EKS in Singapore (ap-southeast-1) across three AZs. Zero downtime is a hard requirement — a failed redemption during a flash sale is direct revenue loss, not just a latency issue.

A. Compute & Architecture

VPC: 10.0.0.0/16 across three AZs. Public subnets for the ALB and NAT gateways only. Private subnets for EKS workers. Database subnets (isolated, no route to internet) for ElastiCache when the app team is ready to deploy it.

NAT: One NAT gateway per AZ. A single NAT saves ~\$65/month but makes the entire cluster dependent on one AZ's network path. Not worth it here.

EKS: Managed control plane in private subnets. Control plane logs (API, audit, authenticator) shipped to CloudWatch. API server has public access enabled for initial deployment — should be locked to a VPN CIDR before production.

VPC Endpoints: S3, ECR API, ECR DKR, STS. This keeps container image pulls and AWS API calls off the NAT gateway, reducing both latency and data processing costs.

Nodes: Baseline group of 3× m6i.xlarge on-demand (one per AZ, always running). Karpenter provisions burst capacity from spot pools (m6i, m6a, m5, m5a) when HPA creates pending pods. Spot is ~70% cheaper than on-demand for the burst window, with on-demand as fallback if spot isn't available.

B. Scalability Strategy

The 10x spike is the core problem. The approach is two layers:

Layer 1 — Pod scaling (HPA): 6 baseline replicas scale up to 60 on CPU (60% threshold) and memory (70% threshold). Scale-up is aggressive: 0-second stabilization window, can double pod count every 60 seconds. Scale-down is conservative: 5-minute cooldown, maximum 10% reduction per minute. The asymmetry is intentional — overshooting on scale-up costs a few dollars, undershooting during a flash sale costs revenue.

Layer 2 — Node scaling (Karpenter): When HPA creates pending pods, Karpenter provisions right-sized spot nodes in ~60 seconds. This is the main reason I chose Karpenter over Cluster Autoscaler — CA requires pre-defined node groups and can't right-size; Karpenter picks the cheapest instance that fits the pending pod's requests. The downside is operational complexity: Karpenter has its own CRDs, IAM setup, and NodePool mental model. Worth it given the burst requirement.

One risk worth calling out: ap-southeast-1 has smaller spot capacity pools than us-east-1 or eu-west-1. During a major APAC flash sale, other services will also be scaling simultaneously. The on-demand fallback handles this, but expect 60-90 seconds of elevated latency while Karpenter switches strategies. Mitigation: the 3 baseline on-demand nodes carry minimum load and don't drain between flash sales.

C. Security & Networking

Network topology: ALB sits in public subnets with WAF attached (rate limit: 5,000 requests per 5 minutes per IP + AWS managed OWASP rules). EKS workers are in private subnets with no direct internet access. All AWS API calls go through VPC endpoints.

Pod-level IAM (IRSA): Each pod assumes its own IAM role via OIDC federation, scoped to redemption-* DynamoDB tables, SQS queues, and Secrets Manager paths. No node-level instance profile with broad permissions — if a container is compromised, the blast radius is limited to the redemption service's own resources.

Network policies: Default deny on both ingress and egress at the namespace level. Explicit allows: ALB → pods on port 8080, pods → AWS services via VPC endpoints (443), pods → Redis on 6379, monitoring namespace → pods for scraping.

KMS: EKS secrets are encrypted at rest with a KMS key (auto-rotation enabled). Application-level field encryption (e.g., PII in DynamoDB) would use a separate key — deferred because the app code is out of scope for this assessment.

TLS: ALB terminates TLS 1.3 (SSL policy ELBSecurityPolicy-TLS13-1-2-2021-06). Internal pod communication is plaintext within the VPC — acceptable given the network policy controls.

D. Reliability & Observability

Surviving an AZ failure: - topologySpreadConstraints with maxSkew: 1 on zone keeps pods spread across AZs - PodDisruptionBudget with minAvailable: 4 — with 6 replicas at 2 per AZ, losing one AZ leaves 4 pods. The PDB prevents anything (rolling update, Karpenter consolidation) from taking it below 4 - DynamoDB and ElastiCache (when deployed) are both Multi-AZ natively

Surviving a bad deployment: - maxUnavailable: 0 on the deployment — rolling update never removes a pod before the new one passes readiness - preStop: sleep 15 — gives the ALB 15 seconds to drain connections before the pod terminates - Three separate probes: startup (generous, allows slow JVM/cold start), readiness (gates traffic), liveness (restarts genuinely hung pods). Having one probe do all three is a common mistake that causes cascading restarts during load spikes

Monitoring: - Structured JSON logs → CloudWatch Logs (90-day retention) - CloudWatch alarms: 5xx error rate > 1% for 3 minutes → SNS → PagerDuty. p99 latency > 500ms for 3 minutes → SNS → Slack - In production I'd add Amazon Managed Prometheus + Grafana for richer dashboards and longer-window analysis. CloudWatch is sufficient for launch and keeps operational complexity down while the team stabilises on EKS

SLOs: Availability 99.95% (~4.4 hours downtime/year), p99 latency < 500ms, error rate < 0.1%

E. Operations

Day 2 toil reduction: - Karpenter expireAfter: 720h rotates nodes every 30 days automatically — no manual patching window - terraform plan runs on every PR, blocking merges if there's drift from expected state - Secrets Manager handles rotation; app reloads via SIGHUP rather than pod restart - CloudWatch alarms alert on the things that actually matter (errors, latency) — not CPU% on individual pods

Team delegation (1 Senior + 2 Juniors, ~3-4 weeks): - **Senior:** VPC, EKS cluster, IAM/IRSA, Karpenter setup, WAF, security review of all manifests before go-live - **Junior 1:** Kubernetes manifests (deployment, HPA, PDB, network policies), staging environment validation, rolling update testing - **Junior 2:** Monitoring setup (CloudWatch log groups, SNS, alarms), runbook authoring, load testing with k6 or Locust to validate the 10x spike scenario

The senior owns anything touching IAM or network topology — mistakes there are hard to recover from. The juniors own the K8s layer where mistakes are visible quickly and rollback is a single command.

Cost Estimate (ap-southeast-1, monthly)

Resource	Est. Cost
EKS control plane	\$73
3× m6i.xlarge on-demand (baseline)	\$430

Resource	Est. Cost
NAT gateways (3×)	\$96
ALB + WAF	\$50
CloudWatch Logs + Alarms	\$35
DynamoDB (on-demand)	\$100
Baseline total	~\$785-850/month

Flash sale burst (Karpenter spot nodes): add ~\$40-120/hour depending on duration and spot price at the time.

Known Gaps

- **Application code not included** — infrastructure only, per assessment scope
- **ElastiCache Redis** — the subnet group and security group are in the VPC module, but the Redis cluster itself is not provisioned. The app ConfigMap has a placeholder endpoint. Deploy order: Terraform → get Redis endpoint from output → inject via External Secrets Operator or kustomize patch before app deploy
- **Prometheus Adapter / custom HPA metrics** — CPU + memory HPA is deployed. Adding an RPS-based metric would require installing the Prometheus Adapter via Helm. Deferred — need real traffic data before tuning custom metrics anyway
- **EKS API server CIDR** — currently `0.0.0.0/0` for convenience. Lock to corporate VPN CIDR or bastion host before production
- **Canary deployments** — rolling update with `maxUnavailable: 0` is the current strategy. Proper canary (progressive traffic shifting) would use Argo Rollouts — worth adding once the team is comfortable with the base setup